



Project Title: AI-Supported Smart Plant Monitoring

Prepared By: Yiğit Kabakcı -212010020092 ,
Berkay Aydoğdu -212010020053

Organization: COMP4901 - Graduation Project I

Date of Issue: January 10, 2026

Status: Version 1.0 Draft

1. Introduction

1.1 Purpose

The purpose of this Software Design Description (SDD) is to provide a comprehensive architectural overview and detailed design specifications for the "AI-Supported Smart Plant Monitoring Stick". This document describes the system's decomposition into subsystems, the data flow between hardware and software components, and the detailed class structures required for implementation. It is intended for software developers, system architects, and testers to ensure the system meets the requirements defined in the Software Requirements Specification (SRS).

1.2 Scope

The "Smart Plant Monitoring Stick" is an IoT ecosystem designed to assist agricultural monitoring through real-time data acquisition and AI-driven analysis. The system comprises three main subsystems:

1. **Embedded Firmware (ESP32):** Responsible for interfacing with capacitive soil moisture sensors, DHT11 temperature/humidity sensors, and photoresistors. It manages power consumption and transmits data via Bluetooth Low Energy (BLE) or Wi-Fi.
2. **Mobile Application (React Native):** Serves as the user interface for pairing devices, visualizing sensor data (graphs/gauges), and receiving notifications. It maintains a local database of plant care profiles.
3. **AI Analysis Module (Backend/Cloud):** Utilizes historical sensor data to predict optimal watering schedules using regression algorithms and provides actionable insights to the user.

1.3 Definitions and Acronyms

- **BLE:** Bluetooth Low Energy.
- **MCU:** Microcontroller Unit (specifically ESP32).

- **UUID:** Universally Unique Identifier (used for BLE Services).
- **JSON:** JavaScript Object Notation.
- **API:** Application Programming Interface.
- **Threshold:** A pre-set value (e.g., minimum moisture level) that triggers an alert.

References

1. **[SRS]** Aydoğdu, B., "Software Requirements Specification for AI-Supported Smart Plant Monitoring Stick," Version 1.0, 2025.
2. **[IEEE 1016]** IEEE Std 1016-2009, "IEEE Standard for Information Technology—Systems Design—Software Design Descriptions."
3. **[FastAPI]** Tiangolo, S., "FastAPI Documentation," <https://fastapi.tiangolo.com/>
4. **[TensorFlow]** Google, "TensorFlow & Keras Documentation," <https://www.tensorflow.org/>
5. **[Flutter]** Google, "Flutter SDK Documentation," <https://docs.flutter.dev/>
6. **[React Native]** Meta Open Source, "React Native Documentation," <https://reactnative.dev/>
7. **[ESP32]** Espressif Systems, "ESP32 Technical Reference Manual," V4.1.
8. **[PlantVillage]** Hughes, D. & Salathe, M., "PlantVillage Dataset," Pennsylvania State University.
9. **[SQLAlchemy]** SQLAlchemy authors, "SQLAlchemy Engine and PostgreSQL Documentation," <https://docs.sqlalchemy.org/>
10. **[Time-Series]** Plotly, "Python Time-Series Documentation," <https://plotly.com/python/time-series/>

3. System Architecture (Composition Viewpoint)

3.1 Architectural Design

The system follows a **Layered Architecture** pattern combined with an **Event-Driven** communication model.

- **Presentation Layer (Mobile App):** Handles user interactions, data visualization, and configuration.
- **Business Logic Layer (App Logic & AI):** Processes raw sensor data, compares it against plant specific thresholds, and generates alerts.
- **Data Layer (Local DB & Cloud):** Stores historical logs and user settings.
- **Hardware Abstraction Layer (HAL - Firmware):** Interfaces directly with physical sensors.

3.2 System Decomposition

The system is decomposed into the following major modules:

A. Firmware Subsystem (ESP32)

- **SensorController:** Periodically polls analog and digital pins to read sensor values. Implements noise filtering algorithms (e.g., moving average) to ensure stable readings.
- **PowerManager:** Manages deep-sleep cycles to extend battery life. Wakes the MCU every 30 minutes for data transmission.
- **CommHandler:** Manages the BLE stack, advertising packets, and GATT (Generic Attribute Profile) server configuration.

B. Mobile Application Subsystem

- **DashboardModule:** Renders real-time charts using libraries like react-native-svg-charts.
- **PlantLibrary:** Contains a static database of 50+ plant types with their ideal environmental conditions (e.g., Tomato: 60-80% moisture).
- **NotificationEngine:** Runs in the background to poll data and trigger local push notifications when critical thresholds are breached.

4. Data Design (Logical Viewpoint)

4.1 Data Entities and Schema

The mobile application uses a local SQLite database for offline capability.

Table: Users

- `user_id` (PK, Integer): Unique identifier.
- `username` (String): User's display name.
- `settings_json` (Text): JSON string storing app preferences (units, theme).

Table: Devices

- `device_mac` (PK, String): MAC address of the ESP32 stick.
- `plant_name` (String): Nickname given by the user (e.g., "Office Fern").
- `plant_type_id` (FK, Integer): Reference to the PlantLibrary.
- `last_sync` (Timestamp): Date of last successful data packet.

Table: SensorLogs

- `log_id` (PK, Integer): Auto-increment ID.
- `device_mac` (FK, String): Reference to the device.

- timestamp (DateTime): Time of reading.
- moisture_pct (Float): Soil moisture percentage (0-100%).
- temperature_c (Float): Ambient temperature in Celsius.
- light_lux (Integer): Ambient light level.

4.2 Communication Protocol (JSON Structure)

Data packet transmitted from ESP32 to Mobile App over BLE Characteristic

```

JSON

{
  "d_id": "A1:B2:C3:D4:E5:F6",
  "bat": 85,
  "data": {
    "m": 45.2,    // Moisture
    "t": 23.5,    // Temperature
    "l": 540      // Light (Lux)
  },
  "ts": 1705423000
}

```

5. Design Viewpoints

5.1 Introduction

This section describes the software design using multiple viewpoints to address the specific concerns of the "AI-Supported Smart Plant Monitoring Stick" project stakeholders. The viewpoints are selected based on IEEE Std 1016-2009 recommendations¹¹¹¹.

+1

5.2 Context Viewpoint

The Context viewpoint depicts the services provided by the design subject with reference to an explicit context².

- **Design Concerns:** System boundary definition, actors, and external environment interactions³.
- **Design Entities (Actors):**

- **User:** Interacts with the system via the Mobile App to view plant health and receive notifications.
- **Plant Environment:** The physical subject providing analog signals (soil moisture, light, temperature).
- **Cloud/AI Service:** External entity providing advanced regression analysis (optional, if offloaded from the app).
- **Interactions:**
 - The **System** acquires raw data from the **Plant Environment**.
 - The **System** transmits processed data to the **User (Mobile App)**.
 - The **Mobile App** alerts the **User** based on AI predictions.

5.3 Composition Viewpoint

The Composition viewpoint describes the way the design subject is structured into constituent parts⁴.

- **Design Concerns:** System decomposition, modular assembly, and component responsibilities⁵.
- **Subsystems:**
 - **Embedded Firmware Subsystem (ESP32):**
 - **Sensor Driver Module:** Handles DHT11 and Capacitive Sensor I/O.
 - **Power Management Module:** Controls Deep Sleep and Wake-up cycles.
 - **BLE Stack:** Manages advertising and GATT server operations.
 - **Mobile Application Subsystem (React Native):**
 - **UI/Presentation Layer:** Dashboard and Settings screens.
 - **Data Persistence Layer:** SQLite database interaction.
 - **Logic Layer:** Threshold comparison and notification triggers.

5.4 Logical Viewpoint

The Logical viewpoint elaborates existing and designed types and their implementations as classes and interfaces⁶.

- **Design Concerns:** Static structure, class hierarchy, and reusability⁷.
- **Key Classes (Firmware - C++):**
 - **SensorManager:** Singleton class responsible for initializing GPIOs and reading values.
 - **BLECommHandler:** Manages the lifecycle of the BLEServer and BLEService.
 - **PowerController:** Interfaces with the RTC (Real Time Clock) to handle sleep modes.
- **Key Classes (Mobile - TypeScript):**
 - **PlantProfile:** Data model defining minMoisture, maxMoisture, and idealTemp.
 - **DeviceManager:** Manages the list of paired BLE MAC addresses.
 - **PredictionEngine:** Contains the logic for the "Time-to-Water" algorithm.

5.5 Dependency Viewpoint

The Dependency viewpoint specifies the relationships of interconnection and access among entities⁸.

- **Design Concerns:** External libraries, hardware coupling, and software prerequisites⁹.
- **Firmware Dependencies:**
 - **BLEDevice.h & BLEUtils.h** (Espressif SDK): Required for Bluetooth functionality.
 - **DHT.h** (Adafruit Unified Sensor): Driver for reading temperature/humidity.
 - **EEPROM.h / Preferences.h:** For non-volatile storage of calibration data.
- **Mobile App Dependencies:**
 - **react-native-ble-plx:** For handling Bluetooth operations on iOS/Android.
 - **victory-native:** For rendering SVG-based data charts.
 - **react-native-push-notification:** For local user alerts.

5.6 Information Viewpoint

The Information viewpoint is applicable for persistent data content¹⁰.

- **Design Concerns:** Data structure, storage schema, and persistence¹¹.
- **Local Database Schema (Mobile App - SQLite):**
 - **Table PLANTS:**
 - id (PK, UUID)
 - name (Text)
 - moisture_threshold (Integer)
 - **Table LOGS:**
 - log_id (PK, Integer)
 - device_id (FK, Text)
 - timestamp (DateTime)
 - moisture_value (Float)
 - temp_value (Float)

5.7 Patterns Use Viewpoint

This viewpoint addresses design ideas as collaboration patterns¹².

- **Design Concerns:** Reuse of architectural styles and design patterns¹³.
- **Applied Patterns:**
 - **Singleton Pattern:** Used in SensorManager (Firmware) to ensure only one instance controls the hardware pins.
 - **Observer Pattern:** Used in the Mobile App's BLE Listener to update the UI automatically when a new notification packet arrives from the stick.
 - **Facade Pattern:** The BLECommHandler provides a simplified interface for the complex underlying BLE stack operations.

5.8 Interface Viewpoint

The Interface viewpoint provides the means to know how to correctly use the services provided by a design subject¹⁴.

- **Design Concerns:** Contract definition, protocol specifications¹⁵.

- **BLE GATT Interface Specification:**
 - **Service UUID:** 4fafc201-1fb5-459e-8fcc-c5c9c331914b
 - **Characteristic: Sensor Data (Read/Notify):**
 - **UUID:** beb5483e-36e1-4688-b7f5-ea07361b26a8
 - **Format:** JSON String {"m": 45, "t": 22}
 - **Characteristic: Configuration (Write):**
 - **UUID:** 1c95d5e3-d8bc-4e31-9965-021e8c753e1a
 - **Payload:** CAL_DRY, CAL_WET, RST.

5.9 Structure Viewpoint

The Structure viewpoint documents the internal constituents and organization of the design subject¹⁶.

- **Design Concerns:** Internal organization of components and execution threads¹⁷.
- **Firmware Execution Structure:**
 - **Core 0:** Reserved for Wi-Fi/BLE Radio stack tasks (FreeRTOS default).
 - **Core 1:** Runs the User Application (loop()), handling sensor polling and data processing.
- **Mobile App Structure:**
 - **Navigation Stack:** Home \$rightarrow\$ DetailView \$rightarrow\$ Settings.
 - **Background Service:** Headless JS task for background data syncing.

5.10 Interaction Viewpoint

The Interaction viewpoint defines strategies for interaction among entities¹⁸.

- **Design Concerns:** Object communication, messaging sequences¹⁹.
- **Sequence: Data Transmission**
 - **Timer** wakes **ESP32** from Deep Sleep.
 - **SensorManager** reads data and passes it to **BLECommHandler**.

- **BLECommHandler** updates the characteristic value.
- **Mobile App** (subscribed) receives the notification.
- **Mobile App** parses JSON and updates **Dashboard**.
- **ESP32** returns to Deep Sleep.

5.11 State Dynamics Viewpoint

Reactive systems and systems whose internal behavior is of interest use this viewpoint²⁰.

- **Design Concerns:** System modes, states, transitions, and reactions to events²¹.
- **ESP32 Power States:**
 - **State A: DEEP_SLEEP:** CPU off, RTC on. Current consumption $< 10\mu\text{A}$.
 - *Transition:* RTC Timer interrupt triggers wake-up.
 - **State B: SENSING:** CPU on, Sensors powered.
 - *Transition:* Data acquisition complete.
 - **State C: ADVERTISING:** BLE radio active.
 - *Transition:* Data sent OR Timeout (30s) reached.
 $\rightarrow$ Go back to State A.

5.12 Algorithm Viewpoint

The Algorithm viewpoint describes the internal details and logic of operations²².

- **Design Concerns:** Computational logic and processing steps²³.
- **Algorithm 1: Soil Moisture Calculation (Firmware)**
 - Collect 10 samples with 10ms delay.
 - Discard min and max values to remove noise.
 - Calculate mean of remaining 8 samples.
 - Map analog value (0-4095) to percentage (0-100%) using calibration offsets.
- **Algorithm 2: AI Watering Prediction (Mobile App)**
 - Input: Time-series array of last 24 moisture readings.
 - Process: Apply Least Squares Linear Regression to find slope (\$m\$).

- Logic: If $m < 0$, calculate time t where $y(t) = \text{Threshold}$.
- Output: Predicted Date/Time for watering.

5.13 Resource Viewpoint

The purpose of the Resource viewpoint is to model the characteristics and utilization of resources²⁴.

- **Design Concerns:** Resource utilization, battery life, memory constraints²⁵.
- **Power Resources:**
 - **Source:** 3.7V 2000mAh Li-Po Battery.
 - **Constraint:** Must operate > 3 months.
 - **Strategy:** 99% Duty Cycle in Deep Sleep. Active time limited to < 1 min/hour.
- **Memory Resources (ESP32):**
 - **Constraint:** 520KB SRAM.
 - **Usage:** JSON buffers restricted to 512 Bytes to prevent stack overflow.
 - **Storage:** 4KB NVS (Non-Volatile Storage) used for persistent calibration settings.

6. Design Rationale

This section outlines the key architectural decisions, the alternatives considered, and the justifications for the selected technologies.

6.1 Hardware Selection Rationale: ESP32 vs. Arduino Uno/Nano

The **Espressif ESP32 SoC** (System on Chip) was selected as the embedded controller instead of standard Arduino platforms (e.g., Uno or Nano). The primary reasons for this decision are:

- **Integrated Connectivity:** The project requires both Wi-Fi and Bluetooth Low Energy (BLE) for data transmission. While Arduino boards require external modules (e.g., HC-05 for Bluetooth, ESP8266 for Wi-Fi), the ESP32 offers these capabilities integrated into a single chip. This reduces hardware complexity, cost, and the physical footprint of the device.

- **Ultra-Low Power Consumption:** Since the "Smart Plant Monitoring Stick" is a battery-operated device, power efficiency is critical. The ESP32's **Deep Sleep** mode reduces current consumption to approximately 10 μ A, allowing the device to operate for months on a single charge. Standard microcontrollers cannot achieve this level of efficiency without significant external circuitry.
- **Dual-Core Architecture:** The ESP32 features a dual-core processor. This allows one core to handle the Wi-Fi/BLE protocol stacks (radio operations) while the second core is dedicated to sensor data acquisition and processing. This separation ensures stable connectivity without interrupting real-time sensor monitoring.

6.2 Mobile Framework Selection Rationale: React Native vs. Native (Swift/Kotlin)

React Native (Cross-Platform) was chosen for the mobile application development instead of developing separate Native applications for iOS (Swift) and Android (Kotlin).

- **Code Reusability and Time Efficiency:** Given the project timeline, React Native allows for the deployment of a single JavaScript/TypeScript codebase across both major mobile platforms. This significantly reduces development and maintenance effort compared to maintaining two separate codebases.
- **Hardware Accessibility:** React Native provides robust access to native device features through libraries such as react-native-ble-plx. This allows the application to manage Bluetooth Low Energy (BLE) connections and data packets with performance comparable to native applications.
- **Rich UI Ecosystem:** The requirement for real-time data visualization (charts and gauges) is efficiently met using established libraries within the React Native ecosystem, such as victory-native or react-native-svg-charts. These libraries facilitate the rapid implementation of the dashboard without the need for building complex UI components from scratch.